

In-Memory Account Ledger Core

Architecture and Trade-offs

Dmitrii Bannikov

10 September 2026

1. Append-only at scale

What breaks first at 100× volume

Not memory. A six-day window of entries is small even at a hundred times the volume, and the log is the cheapest thing in the system to store. What breaks is the *daily close*, and it breaks quadratically.

Every balance in this design is derived by scanning the log for entries with *value_date* $\leq d$. That is $O(N)$ in the number of entries. The close calls it once per account per candidate day, and under the retroactive fee policy the candidate set is every day back to the start of the window, so the close is $O(D \cdot A \cdot N)$. Since N itself grows with volume, multiplying throughput by 100 multiplies close-time work by roughly 10^4 . A close that takes seconds today takes hours, and it takes them inside the overnight window where there is no slack.

The second thing to break is worse, because it is on a latency budget rather than a batch window: the available-balance check on the authorisation path. Deriving a hold's current state means scanning the hold log for that authorisation, and summing active holds means doing that once per authorisation the account has ever had. The cost of approving a card transaction therefore grows with the account's *lifetime* authorisation count, not with the number of holds actually outstanding. An account that has been open a year is slower to authorise than one opened yesterday holding the same funds. That is the wrong quantity to be sensitive to, and it degrades silently until it starts timing out at the scheme.

Where unbounded state accumulates

The log grows without bound, and that is correct: it is the record, and I would not bound it. The defect is not unbounded storage, it is *unbounded read amplification* — three places where the cost of answering a bounded question grows with total history:

- Every balance query reads from the beginning of time. There is no lower bound on the scan, so answering “what is the balance today” gets more expensive every day, forever.
- Settled, released and declined authorisations never leave the hot path. They are dead records that every subsequent approval decision still pays to skip.
- The close-time snapshot map grows one entry per day per account, and nothing ever reads the old ones except a report.

The cheapest change that defers it

In two tiers, because the cheap fix and the right fix are not the same fix.

Immediately, and for almost nothing: maintain a per-(account, value date) delta bucket on append and derive balances by prefix-summing at most D buckets rather than N entries. A backdated entry invalidates only the prefix sums at or after its value date. This turns the close from quadratic in volume into linear in days and touches no semantics, because the index is still *derived* — it can be rebuilt from the log, and a test that asserts the index equals a full replay is a property the design already needs for other reasons. Partitioning holds into open and closed sets is the same kind of change and removes the authorisation-path problem entirely.

Structurally: materialise a cumulative balance checkpoint per account at period boundaries and read *checkpoint plus delta*. The obstacle is that a checkpoint is only sound if the period behind it cannot change, and value-dating is precisely the mechanism by which it can. So the checkpoint is not really an indexing change — it is a policy change wearing an indexing costume, and the policy is a *correction window*: past some cut-off, an entry value-dated into a closed period is refused at intake and rebooked as an adjustment in the open period, carrying the original date as data rather than as position.

That is the change I would argue for first, because it is the only one that pays twice. It bounds the scan, it makes the ledger reconcilable against a general ledger whose periods are already hard-closed, and it bounds the retroactive recomputation that my own failing test exposes — where the fees a customer pays depend on the order corrections arrived in rather than on the contents of the log. One control, two classes of problem.

2. Value-dated entries in production

Operational surface

A value-dated entry is a licence to rewrite a day that has already been reported, reconciled, and in most cases sent to the customer. Everything downstream of a closed day becomes mutable with it.

- **Statement restatement.** A statement is a customer-facing document with legal weight. A backdated entry makes an issued statement wrong, which means the bank needs statement versioning, correction advices, and the ability to reproduce a statement exactly as it was issued — not merely as the ledger reads now. This is why the report in Part 1 keeps both the close-time figure and the restated one; the pair is the minimum needed to explain a charge to the person who received it.
- **Prior-period adjustment.** The general ledger closes on a calendar; the core ledger, as built, does not. A backdated entry that crosses a month end lands in a GL period that no longer accepts postings, and one that crosses a year end touches audited financials. Without an explicit adjustment path the two ledgers simply diverge, and the difference surfaces as an unexplained suspense balance.
- **Retroactive charges.** The highest-risk output of the whole mechanism. The CBUAE Consumer Protection Regulation (Circular 8/2020) and its accompanying Standards require charges to be disclosed, transparent and explainable, with a complaints route. A fee levied for a day whose issued statement showed the account in credit cannot be explained from any document the customer holds.
- **Tax periods.** Explicit fee income carries 5% VAT. A fee dated into an already-filed period changes a filed return.
- **AML and monitoring.** Transaction monitoring aggregates by date; a backdated entry can retrospectively change a threshold aggregation that a structuring rule already evaluated. Under Federal Decree-Law 20/2018 the record must also survive five-year retention in both its original and corrected form — the correction does not replace the original, and the system has to be able to produce both.
- **Internal fraud and earnings management.** Whoever chooses a value date chooses the fee, the interest and the reporting period into which revenue falls. Backdating is a control surface before it is a feature, and in the model as built no event records who chose.

If the licence is Islamic

“A UAE-licensed bank” is not one regulatory perimeter. Onshore, the Higher Shari’ah Authority’s rulings bind every licensed Islamic institution, and the AAOIFI Shari’ah Standards have been mandatory for Islamic banks, Islamic windows of conventional banks and finance companies since 1 September 2018. Inside the Abu Dhabi Global Market, the FSRA applies its own Islamic Finance Rulebook under a common-law framework with its own Shari’a supervisory requirement. The same product gets different answers depending which one it is licensed under.

This matters here because two of the rules I was given do not survive the change, and neither failure is fixable by adjusting a constant.

A daily rate applied to a positive balance is *riba*, and the deeper obstacle is not the interest but the *foreknowledge*: a current account is structured as *qard*, principal guaranteed and return zero, and where a return is paid the account must be *mudarabah* — a share of realised pool profit, declared after the period, with the depositor carrying capital risk. A rate cannot express that. It requires the pool as a dimension of the ledger, average or minimum balance over a period, tenor weightings and smoothing reserves: a different data model, not a different number.

An overdraft fee assessed once per day for as long as a balance stays negative is proportional to the duration of a debt, which is precisely what its flatness disguises. AAOIFI Shari’ah Standard 19 allows recovering the direct actual cost of servicing a *qard* and nothing tied to amount or duration; a late-payment charge is possible under Standards 3 and 8, but as an undertaking to donate — *gharamah* goes to charity, and only *ta’widh*, proven actual loss, may be recognised as income. The ledger consequence is structural: an entry class for non-permissible income that can never reach profit and loss, and a purification account to hold it.

Three things are worth saying in the other direction. The append-only log, value dating and exact remainder distribution are all unaffected. The available-balance rule is arguably more at home under this licence than the conventional one, since an account that cannot go negative is the compliant outcome. And force-posting an unmatched settlement survives: driving the account negative creates an interest-free debt, which is permitted — what is not permitted is charging for the time it stays outstanding.

I implemented the rules as given, because nothing in the brief invokes Islamic finance and refusing a stated rule on my own initiative would be inventing a requirement rather than finding one. The point of raising it is that the licence question has to be asked before the interest and fee logic is designed, not after, and that its answer is not an engineering decision — it belongs to a Shari’ah supervisory committee. Identifying which decisions must leave engineering is part of the design work.

One control before go-live

An enforced value-date correction window with four-eyes approval and mandatory reason coding, applied at the ledger boundary.

An entry whose value date is older than a published cut-off is rejected at intake. Booking one anyway requires a second authorised approver, a reason code drawn from a closed list, and it automatically raises a customer notification whenever the entry changes an issued statement or levies a charge.

I would choose this over the obvious alternatives — a nightly restatement reconciliation report, an alerting threshold on backdated volume, a shadow GL — for three reasons. It is preventive rather than detective, so the exposure never opens rather than being discovered afterwards. It puts a named human against every retroactive charge, which is the first thing both a consumer-protection complaint and a supervisory examination ask for. And it is the same mechanism that makes checkpointing sound in §1, so it is one control answering a regulatory question and a scaling question at once.

3. Authorisation lifecycle

A matching settlement is the least common way an authorisation ends. Every other ending below is reachable in this domain, and each needs a mandated behaviour rather than a default.

One rule governs all of them, and the model as built already follows it: an ending is always an *event*, never a background sweep that mutates state. If available balance rises and no event says why, nobody can answer the customer's question. A second rule follows from the first: what is released is always the *remaining* hold, computed at that moment — never the original amount added back.

Declined at authorisation. Insufficient available balance, a risk decision, or an invalid card state. *Behaviour:* record the decline with its reason and the available balance at that instant, and keep it queryable. Declines are a top complaint category and a fraud signal; a decline that leaves no trace is an unanswerable support ticket. No hold is created.

Expiry without presentment. The acquirer never presents, or presents after the hold window — an abandoned checkout, or a pre-authorisation the merchant simply never uses. *Behaviour:* release automatically at a scheme- and merchant-category-specific window, as an explicit expiry event. A settlement arriving afterwards is not an error; it becomes a force post and is handled as one.

Reversal by the acquirer, full or partial. The terminal cancels, or an estimated authorisation is reduced to the amount actually transacted — the fuel pump that authorises an estimate and reverses the unused portion. *Behaviour:* release exactly the reversed amount and leave the authorisation live at the reduced figure. Treating a partial reversal as a full one over-releases funds; treating it as invalid strands them.

Partial settlement, residual never presented. A split shipment where the remainder is cancelled. *Behaviour:* each settlement posts and decrements the hold; the authorisation ends when the hold reaches zero or the expiry window closes, whichever comes first — not on the first settlement.

Account frozen, closed, or the customer deceased. A sanctions match, a court order, or an estate. *Behaviour:* the hold must not simply be released, because releasing it makes the funds available for withdrawal. Move the held amount to a blocked position and let the eventual settlement or expiry resolve against that, not against the customer's available balance.

Stand-in or offline authorisation the issuer never saw. The scheme approved on the issuer's behalf during an outage, or the terminal authorised below its floor limit. *Behaviour:* there is no authorisation to end — the ledger learns of the obligation only at clearing, and must accept it as a force post rather than reject a payment the scheme has already guaranteed. Matching against the clearing reference, with an amount tolerance, is what separates this from a genuinely bad message.

4. What I cut, and what each cut defers

No intake idempotency. The same event replayed posts twice. Every realistic upstream — a scheme feed, a queue, a retrying client — delivers at least once, so this is a live double-charge defect rather than a theoretical one. It is the first thing I would fix, and it is cheap: a unique constraint on the event identifier, enforced by the store rather than by a read-then-write check.

No concurrency control. Single-threaded, with a sequence counter that is not atomic. Two writers interleaving would make log order stop matching audit order, which quietly destroys the one property the whole design rests on. Production needs a single writer per account or an explicit ordering service — not a lock added later.

Fees derived rather than event-sourced, and the close is not a job. Fee assessment recomputes from current balances instead of being an event carrying the as-of watermark it was computed against, and the day close is a method call rather than a run with an identity, a status and a resume point. These are one defect seen from two sides: it is the flaw the failing test documents — identical entries yield different fees depending on arrival order — and it is also why a close that dies halfway leaves fees half-assessed with no record that it

was attempted. Making each assessment an event attributable to a recorded run fixes both. Deferred risk is a reconciliation break that surfaces only in disaster recovery, which is the worst time to find it.

No actor identity on any event. Nothing records who booked an entry or chose its value date. This defers exactly the control §2 depends on: four-eyes approval is unimplementable on a model that has no notion of eyes.

One settlement per authorisation; no expiry; no partial reversal. Three of the six endings in §3 are unrepresentable. Deferred risk is customer funds held indefinitely against authorisations that will never settle, and split-shipment settlements rejected as duplicates.

No fee-reversal policy. When the entry that caused a fee is reversed, the fee stands, because no policy was given and inventing one silently seemed worse than leaving it visible. Deferred risk is a consumer-protection exposure attached to an undocumented decision — the kind that is discovered by a complaint rather than by a review.

Single-currency fee schedule, no FX. A fee owed on an account whose currency the schedule does not name raises rather than guesses at a rate. Deferred risk is a hard failure on a path that a second currency pair would reach immediately; the fix is a per-currency schedule, not a conversion.

A single licence and a single fee schedule assumed. The interest and fee logic is conventional, and one currency's fee schedule is treated as universal. Deferred risk is that both assumptions are perimeter decisions rather than parameters: an Islamic window or an ADGM licence changes the shape of the interest model rather than its coefficients, and discovering that after the ledger is built is a rewrite, not a migration.

$O(N)$ balance derivation and no checkpointing. Deliberate, to keep the log the single source of truth while the design was still moving. Deferred risk is §1 in full.

Beyond those, the model is layered the way a ledger core has to be if it is to outlive its first product. An *entry* is an account and a signed amount: no dates, no product vocabulary. A *transaction* carries what is true of all its legs — the value date, the booking date, the kind of operation, the event that caused it — and the postings that make it up are validated as a set and appended together. Handlers turn business events into balanced postings.

That boundary is not decoration. It is the test of whether this is a ledger core or a card-payments script: a loan, a term deposit or a standing order should be a new chart entry and a new posting rule, not a new member of an enum inside the core. The ledger does not know what a loan *is*; it knows that some event resolved into postings that balanced. When the operation type lived on the entry instead, the design was already failing that test in a way visible from the code — a contra leg carrying a negative amount labelled CREDIT, a field describing the event while sitting on the posting.

What remains cut is real and chosen. The chart is the minimum that lets each event balance, with no account lifecycle — open, closed, frozen, dormant — and posting rules are written into handlers rather than configured, so a new product still means new code even though it no longer means new core types. Only two balance types exist where production tracks booked, available, cleared, blocked and pending. There is no accounting period, closing entry or statement generation; no reconciliation against an external clearing file and no ageing of the suspense position a force post creates; no dispute or chargeback lifecycle, so everything after settlement is out of scope; no limits or velocity checks beyond the available-balance test; and nothing is published outbound, so no downstream system can learn that any of this happened.

Each is a day of work rather than a rethink, which is why they are cuts and not risks. Each would also need a decision I was not given enough to make.

Sources

Regulatory and Shari'ah claims in §2 rest on: CBUAE, *Higher Shari'ah Authority* and the Rulebook standard on Shari'ah governance; AAOIFI's announcement of the UAE's adoption of its standards (mandatory from 1 September 2018); the ADGM *Islamic Finance Rulebook* and IFR 6.2 on Shari'a Supervisory Boards; the CBUAE *Consumer Protection Regulation* (Circular 8/2020, issued 31 December 2020) and *Consumer Protection Standards*; and AAOIFI Shari'ah Standards 3, 8 and 19 on debt, default and qard. Full URLs, the distinction between what was consulted and what is asserted from prior knowledge, and the limits of the analysis are recorded in `SOURCES.md` in the repository.